Prompt de sistema con salida JSON estructurada (Bloque 10) / Cómo forzar salida JSON y qué hacer si el modelo se resiste

✓ **Hecho:** Ver

Cómo forzar salida JSON y qué hacer si el modelo se resiste

Ya separaste la instrucción de comportamiento de la pregunta del usuario, usando el prompt de sistema.

Esta lección usa exactamente ese mecanismo para resolver un problema muy práctico y muy frecuente: lograr que el modelo responda en un formato JSON estructurado que tu código pueda procesar directamente, en vez de un párrafo de texto libre que tendrías que interpretar a mano.

Prerrequisitos: No necesitas instalar nada nuevo: vas a seguir usando el mismo módulo del Bloque 4, extendido en la lección anterior con "armar_peticion(pregunta_usuario, system_instruction)". Sí vas a agregar dos imports de la biblioteca estándar de Python al principio de tu módulo, "import json" y "import re": "json" ya lo viste de forma conceptual en la Unidad 1.3 (documento 1), y "re" es nuevo, se usa más abajo para limpiar la respuesta del modelo.

📦 Por qué tu código necesita datos, no prosa

Hasta ahora, "extraer_texto" te devuelve exactamente eso: texto.

Sirve para mostrarlo, pero no sirve directamente si tu aplicación necesita, por ejemplo, guardar en una base de datos la categoría de un mensaje, o decidir con una condición "if" qué hacer según el resultado. Intentar sacar esa información de un párrafo con expresiones regulares o buscando palabras clave es fragile: cualquier variación en cómo el modelo redacta la respuesta puede romper tu lógica.

> ✅ **La solución no es procesar mejor el texto libre**: es pedirle al modelo, desde el origen, que la respuesta ya venga en una estructura que tu código pueda leer directamente con **"json.loads"**, sin adivinar nada.

📝 La forma simple: pedirlo explícitamente en el prompt de sistema

El primer nivel, el más básico, es simplemente instruir al modelo con claridad, aprovechando el **prompt** de sistema que ya sabes construir: decirle exactamente qué claves debe tener el **JSON**, y pedirle que no devuelva nada más que eso.

```
system_instruction = (
    "Responde únicamente con un objeto JSON válido, sin texto adicional "
    "y sin usar bloques de markdown. El JSON debe tener exactamente estas "
    "claves: 'categoria' (string) y 'confianza' (número entre 0 y 1)."
)
```

Es común que, incluso pidiéndolo así, el modelo devuelva el **JSON** envuelto en un bloque de markdown, como ```json seguido del contenido y ``` al final.

> 💡 **No lo tomes como una falla total**: es un patrón conocido, y más adelante en esta lección ves cómo manejarlo antes de intentar interpretar la respuesta.

La forma robusta: el modo de salida estructurada de la API

Pedirlo bien redactado ayuda, pero sigue siendo una sugerencia de texto, no una garantía.

La API de Gemini ofrece algo más fuerte: un campo de configuración, "generationConfig", donde puedes indicar que la respuesta debe ser JSON ("responseMimeType"), y opcionalmente describir la forma exacta que ese JSON debe tener ("responseSchema"). Esto no es una sugerencia dentro del texto: es una restricción que la API aplica al generar la respuesta.

Así se ve reflejado en tu código: la misma función "armar_peticion" que extendiste en la lección anterior crece una vez más, esta vez para aceptar, de forma opcional, el esquema exacto que la respuesta debe cumplir.

```
response_schema = {
    "type": "OBJECT",
    "properties": {
        "categoria": { "type": "STRING" },
        "confianza": { "type": "NUMBER" }
    },
    "required": ["categoria", "confianza"]
}

def armar_peticion(pregunta_usuario, system_instruction, response_schema=None):
    api_key = os.getenv("GEMINI_API_KEY")
    modelo = "gemini-3.6-flash"
    url = f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateConter
    body = {
        "systemInstruction": {
            "parts": [ { "text": system_instruction } ]
        },
        "contents": [
            { "parts": [ { "text": pregunta_usuario } ] }
        ]
    }
    if response_schema is not None:
        body["generationConfig"] = {
            "responseMimeType": "application/json",
            "responseSchema": response_schema
        }
    return url, body
```

El parámetro es opcional (por defecto "None") a propósito: cualquier código que ya llame a "armar_peticion" con solo dos argumentos, como el de la lección anterior, sigue funcionando exactamente igual. El nombre y las mayúsculas de "type" ("OBJECT", "STRING", "NUMBER") no son un capricho de formato: son el valor real del enum que usa este endpoint de la API, "models.generateContent". Esto sigue siendo la API REST directa, igual que en toda la unidad: el SDK oficial de Google expresa lo mismo con clases propias, pero el concepto de fondo es idéntico.

Con **"responseSchema"** definido, la **API** ya no te está prometiendo que va a intentar responder en **JSON**: te está devolviendo una respuesta generada bajo esa restricción estructural.

Sigue sin ser una garantía absoluta al cien por ciento en todos los casos, pero es notoriamente más confiable que confiar solo en cómo redactaste la instrucción.



Qué hacer cuando el modelo igual se resiste

Ninguna de las dos formas anteriores elimina por completo la posibilidad de que algo salga distinto a lo esperado: una respuesta cortada, un carácter extra, un bloque de markdown que se coló. Tu código necesita estar preparado para eso, con el mismo tipo de manejo de errores que ya incorporaste en unidades anteriores.

Limpia antes de intentar interpretar

Si el texto viene envuelto en ```` ```json ```` y ```` ``` ````, quítalo antes de pasarlo a `"json.loads"`. Un `".strip()"` y una remoción simple de esas marcas resuelven la mayoría de los casos.

Envuelve el parseo en try/except

`"json.loads"` lanza una excepción si el texto no es JSON válido; ese es exactamente el

tipo de error que ya sabes capturar, para que tu programa no se detenga de golpe.



Valida la estructura, no solo que sea JSON

Un **JSON** válido puede igual venir sin la clave que esperabas. Después de interpretarlo, confirma que las claves requeridas estén presentes antes de usarlas.



Si falla, decide qué hacer, no dejes que falle en silencio

Reintentar la petición una vez, registrar el error con un mensaje claro, o devolver un valor por defecto son todas respuestas válidas; lo que no es válido es que tu programa continúe como si la respuesta hubiera llegado bien cuando no fue así.

```
import json
import re

def parsear_respuesta_json(texto):
    texto_limpio = texto.strip()
    texto_limpio = re.sub(r"^```(?:json)?\s*", "", texto_limpio)
    texto_limpio = re.sub(r"\s*```$", "", texto_limpio)
    try:
        datos = json.loads(texto_limpio)
    except json.JSONDecodeError:
        raise ValueError("La respuesta del modelo no es JSON válido.")
    if "categoria" not in datos or "confianza" not in datos:
        raise ValueError("La respuesta no tiene la estructura esperada.")
    return datos
```

Un detalle que vale la pena que entiendas, no solo que copies: la versión más simple de esta limpieza, con ".removeprefix(```json')" y ".removesuffix(```)", solo reconoce la marca exacta al principio y al final de la cadena. Si el modelo devuelve el bloque delimitado solo con ``` (sin la palabra "json"), o agrega un salto de línea antes de la marca, esa limpieza no la detecta y "json.loads" recibe texto con marcas de markdown todavía adentro. Las expresiones regulares de arriba, con "re.sub", toleran ambas variantes y cualquier espacio en blanco alrededor.

Así se ve todo el flujo junto, de principio a fin, con el esquema y la función de validación que acabas de ver:

```
pregunta_usuario = "Necesito ayuda para resetear mi contraseña, no me llega el correo de ve

url, body = armar_peticion(pregunta_usuario, system_instruction, response_schema)
respuesta_json = enviar_peticion(url, body)
texto = extraer_texto(respuesta_json)
datos = parsear_respuesta_json(texto)

print(datos)
print(type(datos))
```

Con una petición real, algo así (la categoría y el valor de confianza exactos varían según el mensaje y el modelo, pero la forma siempre respeta el esquema):

```
{'categoria': 'soporte_tecnico', 'confianza': 0.88}
<class 'dict'>
```

El segundo "print" no es decorativo: confirma que "datos" ya es un diccionario de Python de verdad, no texto que solo se ve como uno. Esa es, en el fondo, toda la diferencia que construiste en esta lección: pasar de un párrafo que tenías que leer a una estructura que tu código puede indexar directamente, como "datos['categoria']".

Posibles errores comunes

Dos fallas son las más probables en esta lección, y son distintas entre sí. La primera es un **"JSONDecodeError"** capturado por tu propio **"try/except"**: pasa cuando **"texto_limpio"** todavía no es JSON válido después de la limpieza, por ejemplo si la respuesta llegó cortada.

🔎 **Segunda falla:** un error HTTP **400** de la propia API, con el cuerpo **{"error": {"code": 400, "status": "INVALID_ARGUMENT", "message": "...", ...}}**. Pasa cuando **"response_schema"** tiene una forma que la API no puede interpretar, por ejemplo una clave **"type"** con un valor que no es parte del enum real (**"STRING", "NUMBER", "INTEGER", "BOOLEAN", "ARRAY", "OBJECT"**). El mensaje describe qué parte del esquema no pudo leer.

Importante: ninguno de los dos es el error **429** de cuota que ya viste en la Unidad 1.4 ni el **401** de credenciales que ya viste en la Unidad 1.3. Si ves alguno de esos dos, el problema no está en el esquema.



Con esto, ya tienes las dos piezas de esta unidad funcionando juntas: un **prompt** de sistema separado de

la pregunta del usuario, y una salida que llega (o que tu código fuerza a llegar) en una estructura que puedes usar directamente.

Siguiente actividad

Construye tu banco de prompts · Actividad Core

En la última lección de esta **unidad**, la actividad que la cierra, vas a construir un banco con varios de estos **prompts** para tu propia **miniaplicación**, y vas a comprobar con datos que realmente son consistentes, no solo que funcionaron una vez.

Última modificación: jueves, 24 de septiembre de 2026, 17:00